Java Exceptions

Try, Catch, and Custom Handling

codehive

1. The Try...Catch Mechanism

Exceptions are disruptive events that occur during program execution. Exception handling allows your program to "recover" rather than crashing.

```
try {
  // Risky code
  int[] numbers = {1, 2, 3};
  System.out.println(numbers[10]);
} catch (Exception e) {
  // Handle the error
  System.out.println("Error: Index out of bounds!");
}
```

Why use it? Without these blocks, a single mistake like a bad user input would terminate your entire application.

2. Use of Finally Block

The `finally` block is guaranteed to run regardless of whether an exception was thrown or caught. Use it for cleanup tasks like closing files or database connections.

```
try {
  System.out.println(10 / 0);
} catch (Exception e) {
  System.out.println("Cannot divide by zero.");
} finally {
  System.out.println("Cleanup: Releasing resources...");
}
```

3. Manual Triggering with 'throw'

You can force a program to stop and report a specific error using the `throw` keyword. This is helpful for validating business logic.

```
static void validateAge(int age) {
  if (age < 18) {
    throw new ArithmeticException("Minor Restricted");
  } else {
    System.out.println("Access granted!");
  }
}
```

4. Reference: Major Java Exceptions

Exception Class	Common Cause
ArithmeticException	Math issues like division by zero.
ArrayIndexOutOfBoundsException	Attempting to access a non-existent index.
NullPointerException	Attempting to use an object that is 'null'.
NumberFormatException	Converting "Hello" into an Integer.
FileNotFoundException	Accessing a file that isn't on the disk.
InputMismatchException	Passing text when Scanner expects a number.

Best Practices

- ✔ Catch specific exception types rather than the generic `Exception` class whenever possible.
- ✔ Do not leave catch blocks empty; always log or inform the user.
- ✔ Use `finally` to ensure resources are closed properly.
- ✔ Only use `throw` when the program absolutely cannot proceed.

Code Smart. Code Secure.

codehive

Defensive Programming Excellence.